HTTP Fundamentals: Methods, Status Codes, 401 vs 403 & ETags

the details interviewers love

Watch the 3-min video

In one line: HTTP is stateless request → response; idempotent methods are safe to retry, 401 means “who are you?” and 403 “not allowed”, and ETags turn a re-download into a tiny 304.

1 Requests and responses

Every API call you make is HTTP: a request goes out and a response comes back. A **request** has a **method**, a **path**, **headers** and an optional **body**. For example `GET /products/42` with headers like `Host`, `Authorization` and `Accept`. The **response** has a **status code**, headers and a body, like `200 OK` with some JSON.

Request

```
GET /products/42 HTTP/1.1
Host: shop.example.com
Authorization: Bearer eyJhbGci…
Accept: application/json
```

method + path, then headers, then an optional body

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: max-age=3600
ETag: "v7"
{"id": 42, "name": "Lamp"}
```

status code, headers, then the body

Figure 1. One request and its response. Headers say who is calling, what format is wanted and how long the answer may be cached.

HTTP itself is **stateless**: each request stands alone. Cookies or tokens, sent on every request, carry the session, so any server behind a load balancer can handle the next one.

2 Methods: safe and idempotent

Safe means the request doesn't change anything on the server. **Idempotent** means repeating it has the same effect as sending it once. Idempotency is what makes a retry safe after a timeout, when you don't know if the first try worked.

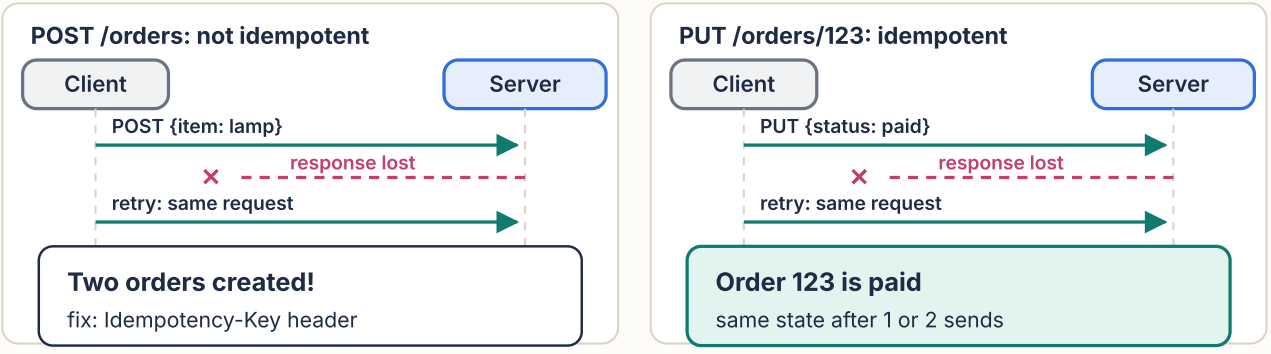


Figure 2. The response to the first send is lost, so the client retries. A second POST creates a second order; a second PUT leaves the same state.

Method	Meaning	Safe	Idempotent
GET	Read a resource	yes	yes
HEAD	Like GET, headers only (check size, ETag, existence)	yes	yes
OPTIONS	Which methods are allowed (used by CORS preflight)	yes	yes
PUT	Replace the resource with this full version	no	yes
DELETE	Remove the resource	no	yes
POST	Create a resource, or run an action	no	**no**
PATCH	Update part of a resource	no	not guaranteed

So a client can safely retry a `PUT`, but retrying a `POST` might create a duplicate order. `PATCH` depends on the change: “set status to paid” is idempotent, “add 1 to quantity” is not.

3 Status codes come in families

Family	Meaning	Common codes
2xx	Success	`200 OK`, `201 Created` (after a POST, with a `Location` header), `204 No Content`
3xx	Redirect, or use your cache	`301` / `302` redirects, `304 Not Modified`
4xx	The client made a mistake	`400 Bad Request`, `401`, `403`, `404 Not Found`, `429 Too Many Requests` (rate limited)
5xx	The server failed	`500 Internal Server Error`, `502 Bad Gateway` (a proxy got a bad answer from upstream), `503 Service Unavailable` (overloaded or down)

The family tells a client what to do. A 4xx won't succeed if repeated unchanged (except 429, after waiting). A 5xx or 429 may be worth retrying with backoff, if the method is idempotent.

4 401 vs 403

A classic interview question. **401 Unauthorized** really means **unauthenticated**: we don't know who you are, so log in or send a valid token (the response includes a `WWW-Authenticate` header saying how). **403 Forbidden** means we know exactly who you are, but you're **not allowed** to do this. A normal user opening an admin page gets a 403. Logging in again won't help.

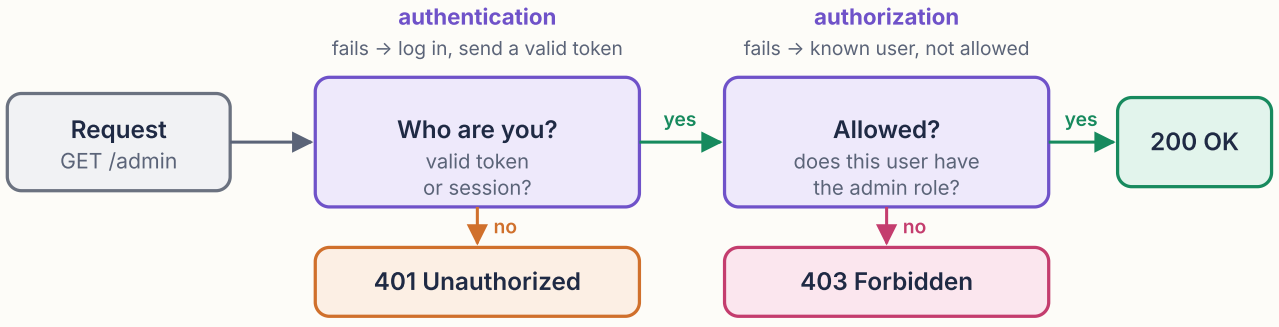


Figure 3. Authentication comes first (fails with 401), then authorization (fails with 403).

5 Caching headers and ETags

Caching headers let browsers and CDNs reuse responses. `Cache-Control` says how long a response stays fresh, like `max-age=3600` (one hour), or `no-store` for private data that must never be cached. **Content negotiation** uses headers too: the client sends `Accept: application/json` or `Accept-Encoding: gzip`, and the server picks the best format it can give and says which in `Content-Type` and `Content-Encoding`.

An **ETag** is a version tag for a response, like `ETag: "v7"`. Next time, the client sends `If-None-Match: "v7"`. If nothing changed, the server replies **304 Not Modified** with no body at all, and the client reuses its cached copy. If it did change, the server sends 200 with the new body and a new ETag.

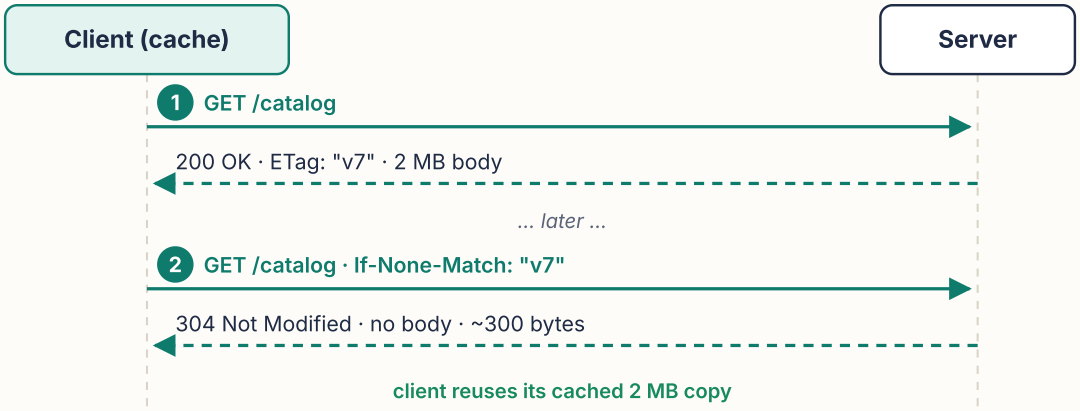


Figure 4. Revalidating with an ETag. The second exchange costs a few hundred bytes instead of 2 MB.

WORKED EXAMPLE: A PRODUCT CATALOG

The catalog response is **2 MB** and changes a few times a day. 100,000 app opens a day, each fetching it: **200 GB** of transfer. With an ETag, unchanged fetches become 304s of about **300 bytes**: 100,000 × 300 B ≈ **30 MB**, plus a few full downloads after each change. Add `Cache-Control: max-age=300` and most opens don't even ask for 5 minutes.

COMMON MISTAKES

- Automatically retrying POST on timeout without an idempotency key: duplicate orders and double charges.
- Returning 401 when the user is logged in but lacks permission (that's 403), or 200 with an error message in the body.
- Changing data in a GET handler. Crawlers, prefetchers and caches assume GET is safe.
- Using `no-cache` when you mean `no-store`. `no-cache` still stores the response, it just revalidates before every use.

6 Key terms

Stateless: each request stands alone

Safe method: doesn't change server state

Idempotent: repeating = same effect as once

401: not authenticated: who are you?

403: authenticated, but not allowed

Cache-Control: how long a response stays fresh

Content negotiation: Accept headers pick the format

ETag: version tag of a response

304 Not Modified: your copy is current, no body

7 Interview questions

Q1. Which HTTP methods are idempotent? BASIC

GET, HEAD, PUT, DELETE and OPTIONS. POST is not, so retries can create duplicates unless you add an idempotency key; PATCH is not guaranteed to be.

Q2. What's the difference between 401 and 403? BASIC

401 means you're not authenticated: send valid credentials. 403 means you're authenticated but not allowed: a normal user on an admin page. Re-logging in fixes a 401, not a 403.

Q3. What does it mean that HTTP is stateless? BASIC

The server keeps no memory between requests; each one carries everything needed, with cookies or tokens identifying the session. That's what lets any server behind a load balancer handle any request.

Q4. A payment POST times out. Can the client retry? SCENARIO

Only safely with an idempotency key: the client sends a unique `Idempotency-Key` header, the server stores the result under that key, and a retry with the same key returns the stored result instead of charging twice.

Q5. Your mobile app downloads a 2 MB catalog on every launch. How do you cut bandwidth? SCENARIO

Return an ETag (and a short `max-age`). The app sends `If-None-Match`; when nothing changed, the server answers 304 with no body, about 300 bytes instead of 2 MB. Also serve it gzip-compressed via `Accept-Encoding`.

Q6. Users get 502 from the load balancer, and 503 at peak. What do they tell you? SCENARIO

502: the load balancer got an invalid response from a backend (it crashed or closed the connection). 503: the service is overloaded, has no healthy backends, or is in maintenance; clients should back off, ideally honouring `Retry-After`.

Q7. DELETE is idempotent, but the second DELETE returns 404. Is that a contradiction? TRICKY

No. Idempotency is about the effect on server state, not the response. After one or two DELETEs the resource is gone either way; the status code can differ.

Q8. Is PATCH idempotent? TRICKY

Not guaranteed. A PATCH that sets fields (`{"status": "paid"}`) is idempotent; one that applies a relative change ("add 1 to quantity") is not. Treat PATCH like POST for retries unless your API defines it otherwise.

Next: SD-B-011 · HTTPS & TLS 1.3 backendroadmap.com